On Top-k Retrieval for a Family of Non-monotonic Ranking Functions*

Nicolás Madrid[1] and Umberto Straccia[2]

[1] Centre of Excellence IT4Innovations, Division of the University of Ostrava, Institute for Research and Applications of Fuzzy Modeling, Czech Republic
nicolas.madrid@osu.cz
[2] Istituto di Scienza e Tecnologie dell'Informazione (ISTI - CNR), Pisa, Italy
straccia@isti.cnr.it

Abstract. We presented a top-k algorithm to retrieve tuples according to the order provided by a non-necessarily monotone ranking funtion that belongs to a novel family of functions. The conditions imposed on the ranking functions are related to the values where the maximum score is achieved.

1 Introduction

Usually when users make queries in databases, they are only interested in a subset of answers. Consider just a search for a house in a database according to some preferences about size, location, etc. In such a case, an user usually is not interested in knowing which is the worst house according to his preferences: an answer with simply the ten best houses is good enough. Top-k algorithms deal with that issue and have become an important topic of interest in the last years [1,2,5,6,8,9]. Roughly speaking, the answer of a top-k query is the subset with the k best results.

It is worth mentioning that, a priori, top-k algorithms can be defined on various and diverse frameworks; for instance on *fuzzy logic programming* [9] and on *uncertain databases* [11,12]. However, most approaches of top-k retrieval have been developed on relational databases [3,4]; this paper is not an exception. Hence, for us, an answer of a top-k query is a set of k tuples with the greatest score (according to a ranking function f) among those which satisfy a relation R (called *the joint condition*).

The typical procedure to obtain the answer of a top-k query consists in developing a *threshold algorithm* which computes an upper bound for the scores of tuples non retrieved yet. Usually, threshold algorithms need two requirements: firstly a database sorted in decreasing order with respect to score; and secondly, a monotonic ranking function.

In this paper instead of requiring monotonic mappings, we will consider a most general family of ranking functions. Note that removing the monotonicity

* This work was supported by the European Regional Development Fund projects CZ.1.05/1.1.00/02.0070 and CZ.1.07/2.3.00/30.0010.

H.L. Larsen et al. (Eds.): FQAS 2013, LNAI 8132, pp. 507–518, 2013.
© Springer-Verlag Berlin Heidelberg 2013

as a requirement in ranking functions arises naturally in many contexts. Just consider an user searching for a house who is interested preferably in houses with a size close to 100 m^2. In such a case the function determining how much far is the size of a house from 100 m^2 is obviously not monotonic.

Allowing the use of non monotonic ranking functions in top-k algorithms is a current challenge. To the best of our knowledge, there are only two papers dealing with non-monotonic ranking functions; namely [7] and [10]. In [10], the authors require an indexed-merge structure in the database and the procedure consists in partitioning the domain in sub-domains where the ranking function (a priori arbitrary) is monotonic (or semi-monotonic). On the other hand, [7] defines the top-k procedure by considering *isolines* in a specific family of ranking functions.

The approach described in this paper is the first step of a more general research towards the definition of a top-k algorithm for arbitrary ranking functions. Specifically, in this paper we present a top-k procedure for a family of ranking functions allowing the use of distance functions among others.

In the following, we proceed as follows. In Section 2 we present the properties that a ranking function has to verify. Moreover, we provide some additional properties and methods to construct them. In Section 3 we describe our top-k algorithm and give the theorem of correctness of the procedure. Finally in Section 4 we present the conclusions and address future work.

2 Ranking Functions

A *ranking function* is defined as follows.

Definition 1. *A mapping* $f\colon [0,1]^n \to \mathbb{R}$ *is called a* ranking function *if there exists an element* $(m_1, \ldots, m_n) \in [0,1]^n$ *such that for all* $(c_1, \ldots, c_n) \in [0,1]^n$ *and all* $i \in \{1, \ldots, n\}$*, the mapping defined by:*

$$g_i(x) = f(c_1, \ldots, c_{i-1}, x, c_{i+1}, \ldots, c_n)$$

is monotonic in $[0, m_i]$ *and antitonic in* $[m_i, 1]$.[1]

Let us explain briefly the condition imposed on ranking functions. The tuple $(m_1, \ldots, m_n)$ (called the *best preference* of f) represents the scores associated to the best possible answer ranked by f. Hence, the coefficients $m_1, \ldots, m_n$ can be interpreted as a preference for tuples with such scores. Moreover, those coefficients do not represent simply "an overall best preference" but also *"local best preferences"*, since the closer the i-th variable to m_i, the greater the value of f; independently of the values in the rest of variables.

The family of ranking functions contains some interesting families as the set of monotonic and antitonic mapping defined from $[0,1]^n$ to $\mathbb{R}$.

Proposition 1. *Any* monotonic mapping *(resp.* antitonic mapping*) is a ranking function.*

[1] For the sake of clarity, we recall that a mapping $f\colon [0,1] \to \mathbb{R}$ is monotonic (resp. antitonic) if $x \leq y$ implies $f(x) \leq f(y)$ (resp. $f(x) \geq f(y)$) for all $x, y \in [0,1]$.

The following example shows that our notion of ranking functions can deal, somehow, with distances as well.

Example 1. Let d be the Euclidean distance, then the mapping defined by:

$$f((x_1, x_2)) = \sqrt{2} - d((x_1, x_2), (1/2, 1/2)) = \sqrt{2} - \sqrt{(1/2 - x_1)^2 + (1/2 - x_2)^2}$$

is a ranking function, whose maximal score is reached at $(1/2, 1/2)$.

Actually, the family of distances induced by norms can be considered as ranking functions as shown in the following proposition.

Proposition 2. *Let $d\colon [0,1]^n \times [0,1]^n \to \mathbb{R}^+$ be a distance induced by a norm and let $(m_1, \ldots, m_n) \in [0,1]^n$. Then the mapping defined by:*

$$\begin{aligned} f\colon [0,1]^n &\to \mathbb{R} \\ f(x_1, \ldots, x_n) &= -d((x_1, \ldots, x_n), (m_1, \ldots, m_n)) \end{aligned}$$

is a ranking function whose maximal score is reached at $(m_1, \ldots, m_n)$.

Proof. To prove that f is a ranking function we have to show that for all $(c_1, \ldots, c_n) \in [0,1]^n$, each mapping g_i defined by

$$g_i(x) = -d((c_1, \ldots, c_{i-1}, x, c_{i+1}, \ldots, c_n), (m_1, \ldots, m_{i-1}, m_i, m_{i+1}, \ldots, m_n))$$

is monotonic on $[0, m_i]$ and antitonic on $[m_i, 1]$.

Without lost of generality we assume that $i = 1$. Moreover, we use a well known result of metric space theory: if d is a distance induced by a norm, then the closed ball of radius $r > 0$ centered at x:

$$B(x, r) = \{y \in X\colon d(y, x) \leq r\}$$

is convex (i.e. all line segment bounded by two points of $B(x, r)$ is contained in $B(x, r)$).

Now, consider $y \leq x \leq m_1$ and let us show that $g_1(y) \leq g_1(x)$; that is

$$-d((y, \ldots, c_n), (m_1, \ldots, m_n)) \leq -d((x, \ldots, c_n), (m_1, \ldots, m_n))$$

or equivalently:

$$d((y, \ldots, c_n), (m_1, \ldots, m_n)) \geq d((x, \ldots, c_n), (m_1, \ldots, m_n))$$

Note that

$$d((y, \ldots, c_n), (m_1, \ldots, m_n)) = d((2m_1 - y, \ldots, c_n), (m_1, \ldots, m_n))$$

since:

– $d((y, \ldots, c_n), (m_1, \ldots, m_n)) = ||(m_1 - y, \ldots, m_2 - c_n)||$

- $d((2m_1 - y, \ldots, c_n), (m_1, \ldots, m_n)) = ||(2m_1 - y - m_1, \ldots, m_2 - c_n)|| =$
  $= ||(m_1 - y, \ldots, m_2 - c_n)||$.

Now, consider $r = d((y, \ldots, c_n), (m_1, \ldots, m_n))$. Then both $(y, \ldots, c_n)$ and $(2m_1 - y, \ldots, c_n)$ belongs to $B((m_1, \ldots, m_n), r)$. Additionally, note that $(x, \ldots, c_n)$ belongs to the line segment bounded by $(y, \ldots, c_n)$ and $(2m_1 - y, \ldots, c_n)$ since $y \leq x \leq m_1 \leq 2m_1 - y$. Thus, by the result presented above, $(x, \ldots, c_n) \in B((m_1, \ldots, m_n), r)$; or equivalently:

$$d((x, \ldots, c_n), (m_1, \ldots, m_n)) \leq r = d((y, \ldots, c_n), (m_1, \ldots, m_n)) \ .$$

The previous result allows us to use the idea *"the closer, the better"* in ranking functions. The following result has the aim of facilitating the obtainment of ranking functions.

Proposition 3. *Let $f_1 \colon [0,1]^n \to [0,1]$ and $f_2 \colon [0,1]^k \to [0,1]$ be two ranking functions and let $g \colon [0,1]^2 \to [0,1]$ be a monotonic function. Then the function defined by: $f \colon [0,1]^n \times [0,1]^k \to [0,1]$*

$$f(x, y) = g(f_1(x), f_2(y))$$

is a ranking function. Moreover, if $n = k$ and the best preferences of f_1 and f_2 coincide, then the function defined by: $\overline{f} \colon [0,1]^n \to [0,1]$

$$\overline{f}(x) = g(f_1(x), f_2(x))$$

is a ranking function as well.

Proof. The proof is straightforward just taking into account that if m_1 and m_2 denote the best preferences of f_1 and f_2, then (m_1, m_2) is the best preference of f. Moreover, in the case that both best preference coincide, then it is easy to prove that m_1 (or equivalently m_2) is the best preference of $\overline{f}$.

Note that although the proposition above has been defined to deal with two ranking functions, it easily extensible to deal with a numerable amount of ranking functions. Moreover, as a consequence, compositions of ranking functions with monotonic mappings are also ranking functions.

Corollary 1. *Let $f \colon [0,1]^n \to [0,1]$ be a ranking functions and let $h \colon [0,1]^2 \to [0,1]$ be a monotonic function. Then the function defined by $h(f(x))$ is a ranking function.*

Proof. Just apply Proposition 3 to functions $f_1 = f_2 = f$ and $g = h \circ p_1$; where $p_1 \colon [0,1]^2 \to [0,1]$ denotes the projection on the first component.

Example 2. It is easy to show that the mapping provided in Example 1 is a ranking function by using Proposition 2 and Corollary 1. Consider the mappings $f_1 \colon [0,1]^2 \to \mathbb{R}$ and $f_2 \colon \mathbb{R} \to \mathbb{R}$ given by

$$f_1(x_1, x_2) = -\sqrt{(1/2 - x_1)^2 + (1/2 - x_2)} \qquad f_2(x) = \sqrt{2} + x$$

Note that f_1 is the negated Euclidean distance from the point $(1/2, 1/2)$, so we can assert that f_1 is a ranking function (Proposition 2). On the other hand, f_2 is monotonic, so by Corollary 1, f is a ranking function. Finally, the mapping f given in Example 1 is the composition of f_1 and f_2; i.e. $f(x_1, x_2) = f_2(f_1(x_1, x_2))$.

Example 3. The mapping given by:

$$f(x_1, x_2) = \begin{cases} 2 & \text{if } x \geq 1/2 \text{ and } y \geq 1/2 \\ 2 - \sqrt{(x_1 - 1/2)^2 + (x_1 - 1/2)^2} & \text{otherwise} \end{cases}$$

is a ranking function since is the maximum of the following two ranking functions

$$f_1(x_1, x_2) = \begin{cases} 2 & \text{if } x \geq 1/2 \text{ and } y \geq 1/2 \\ 0 & \text{otherwise} \end{cases} \quad f_2(x_1, x_2) = 2 - \sqrt{(x_1 - 1/2)^2 + (x_1 - 1/2)^2}$$

with the same best preference ($m = (1/2, 1/2)$). That is

$$f(x_1, x_2) = \max\{f_1(x_1, x_2), f_2(x_1, x_2)\} \ .$$

Below we present some properties of our ranking functions. The first result shows that the best preference is the point where the maximum is reached.

Proposition 4. *Let $f\colon [0,1]^n \to \mathbb{R}$ be a ranking function and let $m \in [0,1]^n$ be its best preference. Then:*

$$\max_{x \in [0,1]^n} f(x) = f(m)$$

The next Lemma is slightly more general than the previous Proposition.

Lemma 1. *Let $f\colon [0,1]^n \to \mathbb{R}$ be a ranking function with best preference $(m_1, \ldots, m_n)$. Then for all $c \in [0,1]$,*

$$\max_{(x_1, \ldots, x_n) \in [0,1]^{n-1}} f(x_1, \ldots, x_{i-1}, c, x_{i+1}, \ldots, x_n) = f(m_1, \ldots, m_{i-1}, c, m_{i+1}, \ldots, m_n) \ .$$

Proof. Let us show that for all $(a_1, \ldots, a_{i-1}, c, a_{i+1}, \ldots, a_n) \in [0,1]^n$ we have:

$$f(a_1, \ldots, a_{i-1}, c, a_{i+1}, \ldots, a_n) \leq f(m_1, \ldots, m_{i-1}, c, m_{i+1}, \ldots, m_n) \ .$$

Note that from the above inequality, the result is straightforward. Let us assume that $a_1 \in [0, m_1]$ (resp. $a_1 \in [m_1, 1]$). By using that f is a ranking function, we have that the mapping defined by $g(x) = f(x, a_2 \ldots, a_{i-1}, c, a_{i+1}, \ldots, a_n)$ is monotonic on $[0, m_1]$ (resp. antitonic on $[m_1, 1]$); thus we have the inequality:

$$f(a_1, \ldots, a_{i-1}, c, a_{i+1}, \ldots, a_n) \leq f(m_1, \ldots, a_{i-1}, c, a_{i+1}, \ldots, a_n) \ .$$

By using the same reasoning inductively for $a_2, \ldots, a_n$, we obtain eventually:

$$f(a_1, \ldots, a_{i-1}, c, a_{i+1}, \ldots, a_n) \leq f(m_1, \ldots, m_{i-1}, c, m_{i+1}, \ldots, m_n) \ .$$

The following result plays an important role in the correctness of the top-k algorithm described in the next section.

Lemma 2. *Let $f\colon [0,1]^n \to \mathbb{R}$ be a ranking function with best preference $m = (m_1, \ldots, m_n)$ and let $I = [a_1, b_1] \times \cdots \times [a_n, b_n] \subset [0,1]^n$ be an interval satisfying that $m \in I$. Then for all $x \in [0,1]^n \setminus I$ we have:*

$$f(x) \leq \max\big\{f(a_1, m_2, \ldots, m_n), f(m_1, a_2, \ldots, m_n), \ldots, f(m_1, m_2, \ldots, a_n), \\ f(b_1, m_2, \ldots, m_n), f(m_1, b_2, \ldots, m_n), \ldots, f(m_1, m_2, \ldots, b_n)\big\} \ .$$

Proof. Consider $x = (x_1, \ldots, x_n) \in [0,1]^n \setminus I$, then at least one component x_i has to be either lesser or equal than its respective a_i or greater or equal than its respective b_i. Let us assume without lost of generality that $x_1 \leq a_1$. Then, by Lemma 1:

$$\max_{(y_2, y_3, \ldots, y_n) \in [0,1]^{n-1}} \{f(x_1, y_2, \ldots, y_n)\} = f(x_1, m_2, \ldots, m_n) \ .$$

As a particular case we obtain that:

$$f(x_1, x_2, \ldots, x_n) \leq f(x_1, m_2, \ldots, m_n) \ .$$

So, thanks to the monotonicity of $f(x, m_2, \ldots, m_n)$ in $[0, a_1] \subseteq [0, m_1]$ and that $x_1 \in [0, a_1]$ we conclude:

$$f(x_1, x_2, \ldots, x_n) \leq f(x_1, m_2, \ldots, m_n) \leq f(a_1, m_2, \ldots, m_n) \ .$$

3 Top-k Retrieval Algorithm

In this section we describe a top-k retrieval algorithm[2] for query answering over relational databases, where tuples are ranked by a function belonging to the family of ranking functions introduced in Section 2. The algorithm presented in this paper is based on [3], which presents a top-k retrieval algorithm for monotonic ranking functions. However, the presence of a more general family of ranking functions than the monotonic functions one requires considering a different tuple retrieval strategy.

3.1 Join Strategy

The join strategy used to retrieve tuples plays an important role in top-k algorithms, since it is crucial for the correctness and response time. Note that if tuples associated to the best preference have not been considered yet, we can not ensure that we have already retrieved the top-1. So, it seems appropriated to check the join condition from the tuples which scores are associated with the best preference instead of from the top. Somehow, the idea underlying in this alternative is: *the closer from the best preference, the better are the results.*

The main drawbacks of this strategy with respect to the strategies starting from the top are two. On the one hand, for each table we must consider now

[2] For lack of space, it is not possible to describe formally the top-k problem. So the reader is referred to [3].

two directions to retrieve tuples: up and down (or equivalently decreasing and increasing the scores from the start point). So this increments slightly the storage cost of the procedure. On the other hand, before applying the algorithm, we need to arrive somehow to the tuples associated to the best preference. Just note that the computational cost of this pre-procesing is aceptable ($O(log\ n)$).

The join strategy used in the top-k described in this paper generalizes the *"symmetric Ripple Join strategy"* given in [3]. In the case of making joins with only two tables A and B, the difference between both strategies can be described easily. That is because we can represent graphically how both strategies sweep the cartesian product both table's scores $X.A \times X.B$. Roughly speaking, the difference between both approaches is that whereas [3] sweeps the cartesian plane from a corner, our approach sweeps the cartesian plane from a point allocated, a priori, anywhere.

Figure 1 below represents five steps of the join strategy by considering a *"clockwise movement"*. Suppose that the best preference of the weak ranking function f is (a, b). The first iteration simply gets two tuples, one from A and another from B with scores a and b respectively and checks the join condition for both tuples. In the case there is not tuples with scores a in A (resp. b in B), we insert a new tuple with score a (resp.b) in A (resp. in B) non satisfying the join condition with any tuple. The second step consists in considering the tuple of A with the closest score to a between the tuples non considered yet and with a score less or equal than a. So the new tuple considered in A is achieved by decreasing the score. Before to pass to the next step, it is necessary to verify if the new tuple considered in A holds the join condition with the only tuple considered in B. In the third step we retrieve a new tuple of B, as in the previous step, by choosing a tuple decreasingly. In other words, we consider the tuple in B with the closest score to b between the tuples non considered yet and with a score less or equal than b. Additionally, in this step we verify the join condition between all posible combination of tuples considered up to this step. The fourth and fifth steps are similar to the second and third steps respectively but by considering greater scores. That is, firstly it is considered the tuple in A (resp. B) with the closest score to a (resp. b) between the tuples non retrieved yet and with a score greater or equal than a (resp. b); secondly it is verified the join condition for all posible combination between the tuples already considered in B (resp. A).

We provide two remarks to finish this section. Firstly, the Ripple Join can be considered as a particular case of the Clockwise Join; specifically when the best preference coincides with the point $(1, 1)$. Secondly, it is possible to consider asymmetric strategies following the idea underlying in the clockwise join. That is, instead of retrieving tuples one by one (either one of A by one of B or one increasingly by one decreasingly), we can use heuristic to determine a preferential direction.

3.2 Top-k Retrieval Algorithm

The algorithm is divided in two procedures, the main source $Top.k.retrieval$ and the subroutine $Get.Next.Tuple$. Briefly, $Get.Next.Tuple$ retrieves in each

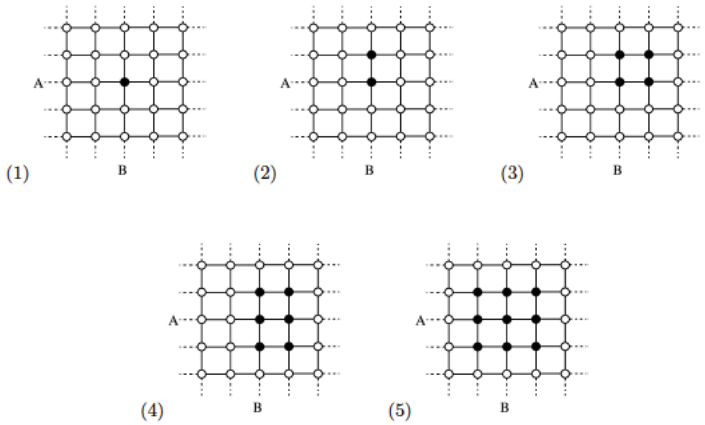

Fig. 1. Our join strategy

step the best result between the tuples non retrieved yet and $Top.k.retrieval$ determines when $Get.Next.Tuple$ can stop of retrieving tuples. To facilitate the presentation, both algorithms are defined and described here by considering only two tables; however it can be generalized to an arbitrary number of tables.

For the sake of clarity we describe the variables appearing in $Top.k.retrieval$:

- $Top.Tuple.List$: in this list we include one by one the top-k results. Actually, this list is the output of the algorithm.
- Q: this list contains temporally each tuple retrieved by using the join condition but non included in $Top.Tuple.List$ yet.
- $firstTuple.A$ and $firstTuple.B$: these Boolean variables are used in the subroutine $Get.Next.Tuple$ to get in buffer the variables $A.init$ and $B.init$; where the value of the best preference is saved.
- tuple: is the answer of $Get.Next.Tuple$. Such variable can be either a tuple belonging to the top-k or the value "NULL" if no more tuple can be obtained by the join condition.

The procedure $Top.k.retrieval$ works as follows. Initially the lists $Top.Tuple.List$ and Q are empty and the Boolean variables $firstTuple.A$ and $firstTuple.B$ are considered to be $true$. In each loop, a call to the procedure $GetNextTuple$ is done. The latter returns the tuple with the best score between the tuples satisfying the join condition and are not already in the list $Top.Tuples.List$. The loop is broken only in two cases: either if we have already retrieved k tuples by using the subroutine $GetNextTuple$ (step 4) or if it is impossible to retrieve more tuples by using the join condition (step 6).

The variables appearing in $GetNextTuple$ and not in $Top.k.retrieval$ are described as follows:

- $Top.Q$: gets the tuple in Q with the best score.

```
Procedure: Top.k.retrieval(A,B,R,k,f)
output: the list Top.Tuples.List with the k best tuples
inputs: A,B: two ranked relational tables
    R: a relation used in the join condition
    k: number of best tuples we are interested in
    f: weak ranking function
init:
  firstTuple.A= true;
  firstTuple.B= true;
  Top.Tuples.List = ∅;
  Q = ∅;
1.  Loop
2.    tuple := GetNextTuple;
3.      include tuple in Top.Tuples.List;
4.      if (|Top.Tuples.List| = k) then
5.        break Loop;
6.      if (tuple = NULL) then
7.        break Loop;
8.  end Loop;
9.  end;
```

Fig. 2. Top.k.retrieval Algorithm

- $A_{\uparrow}$; $A_{\downarrow}$;$B_{\uparrow}$; $B_{\downarrow}$: are the scores of the respective last tuples seen in each table w.r.t. each direction (see Section 3.1).
- $A.init$; $B.init$: are the scores of the respective first tuple retrieved by each table. Note that those values represent the best preference of the ranking function.
- T: this value is a threshold which upper bounds the score of the rest of tuple non retrieved yet.
- $nextTuple$: this Boolean variable is used to indicate when it is imposible to retrieve more tuple satisfying the join condition.

The procedure $GetNextTuple$ works as follows. Previously to start with the loop, the algorithm checks if there is already any tuple in Q with a score greater or equal than the threshold T; or equivalently if the top-1 of the tuples non contained in $Top.Tuple.List$ belongs to Q. In such a case, is not necessary to activate the loop and the procedure ends (step 5). Otherwise (as in the first call, since Q is empty) the procedure activates a loop. That loop generates tuples till one answer can be returned. Two cases are considered to generate new tuples:

- First of all, if every tuple in tables A and B has been already considered, then no more tuples can be generated. In such case the Boolean variable $nextTuple$ gets the value $false$ and the algorithm goes directly to step 21.
- Otherwise, we consider one new tuple (either from A or from B depending on the joint strategy considered). If this new tuple is the first tuple considered in the table (step 11), then the score of such tuple is one of the coefficients of the best preference; so this value is saved in the variable $I.init$. Anyway, the loop between the steps 15 and 20 recomputes the threshold T, determines all possible join combinations and includes the new tuples in Q decreasingly by using the ranking function f.

Between the steps 20 and 32 the algorithm computes the answer to return to $Top.k.retrieval$. Such answer depends on which case holds:

- If there exists in Q a tuple with score greater or equal than T (step 20) then the answer to return is the tuple with the greatest score in Q.
- If the algorithm has retrieved the whole set of tuples satisfying the joint condition and the list Q is not empty (step 26), the algorithm returns the tuple in Q with the greatest score.
- Finally, if every tuple of A and B has been considered and Q is empty, the algorithm returns "NULL" . This answer means that no more results can be obtained by the join condition, so it is imposible to return the k better results since there is no k tuples satisfying the join condition. With this answer, the main procedure $Top.k.retrieval$ breaks its loop and ends.

```
Procedure: GetNextTuple
output: Next tuple to attach in Top.Tuples.List
init: NextTuple=true;
1.   if (Q ≠ ∅) then
2.     tuple = Top.Q
3.     if (tuple.score ≥ T) then
4.       return tuple;
5.       end;
6.   Loop
7.     Determine the next seen tuple, I* (Comment: I* ∈ {A↑, A↓, B↑, B↓});
8.     if (No next seen tuple I* can be considered) then
9.       nextTuple = false ;
10.    else
11.      if (firstTuple.I = true) then
12.        I.init = I*.score
13.        firstTuple.I = false
14.      I*.lastSeen = I*.score
15.      T = max(f(A.init, B↑.lastSeen), f(A.init, B↓.lastSeen),
16.              f(A↑.lastSeen, B.init), f(A↓.lastSeen, B.init));
17.      determine all possible join combination;
18.      For each valid join combination by using the relation R
19.        compute the score result by using f;
20.        insert each join result in Q and rank them;
21.    if (Q ≠ ∅) then
22.      tuple = Top.Q
23.      if (tuple.score ≥ T) then
24.        remove tuple from Q;
25.        break Loop;
26.      else
27.        if ( nextTuple = false ) then
28.        remove tuple from Q;
29.        break Loop;
30.    else
31.      if ( nextTuple = false ) then
32.        tuple = NULL;
33.        break Loop;
34.  end Loop;
35.  return tuple;
36.  end;
```

Fig. 3. GetNextTuple Algorithm

3.3 Correctness of the Algorithm

At follows we attend to the correctness of the algorithm.

Lemma 3. *The algorithm GetNextTuple correctly reports the best join result (according to the ordering provided by f) among the tuples which do not belong to the set Top.Tuple.List and satisfy the join condition.*

Proof. Let x_A and x_B be the scores in A and B associated to the tuple returned by *GetNextTuple*. By definition (step 22) the score of such tuple ($f(x_A, x_B)$) is greater or equal that any score of the tuples belonging to the list Q. Note as well that, by the join strategy, the scores in A (resp. B) associated to any tuple in Q belongs to the interval $[A_\downarrow, A_\uparrow]$ (resp. $[B_\downarrow, B_\uparrow]$).

Let $\overline{x}_A$ and $\overline{x}_B$ be the scores in A and B associated to a tuple satisfying the join condition but non retrieved yet; and therefore non belonging neither to *Top.Tuple.List* nor to Q. Then, by the join strategy, the tuple $(\overline{x}_A, \overline{x}_B)$ belongs to $[0,1]^2 \setminus [A_\downarrow, A_\uparrow] \times [B_\downarrow, B_\uparrow]$. Applying now the Lemma 2, we can assert that the score of such a tuple is upper bounded by:

$$f(\overline{x}_A, \overline{x}_B) \leq \max\left\{f(A_\downarrow, B.init); f(A_\uparrow, B.init); f(A.init, B_\downarrow); f(A.Init, B_\uparrow)\right\} = T$$

Now, (by step 23) the score of the tuple returned by *GetNextTuple* holds necessarily $f(\overline{x}_A, \overline{x}_B) \leq T \leq f(x_A, x_B)$. In conclusion, the score of the tuple returned by *GetNextTuple* is greater or equal that the score of any tuple in Q and any tuple non retrieved yet; in other words, the score of such a tuple has the greatest score between the join results non belonging to *Top.Tuple.List*.

Theorem 1. *Let A and B be two relational tables, R be a binary relation, k be a natural number and f be a ranking function. Then $Top.k.retrieval(A, B, R, k, f)$ correctly reports the top-k join results ordered by f.*

Proof. The proof comes directly from the Lemma 3, since in each step the loop *GetNextTuple* inserts in *Top.Tuple.List* the tuple with the greatest score among the tuples non belonging to *Top.Tuple.List* and satisfying the join condition.

4 Conclusion and Future Work

We have presented a top-k algorithm to retrieve tuples according to the order provided by a non-necessarily monotone ranking function that belongs to a novel family of functions satisfying some conditions related to the values where the maximum is achieved. For instance, this approach is the first one that allows the use of the mapping $f : [0,1]^2 \to \mathbb{R}$ given by $f((x,y) = e^y - (x - 0.5)^2$ as a ranking function in top-k retrieval over non-index-merge paradigms.

The ultimate goal of our research is to develop a top-k algorithm for queries ordered by arbitrary mappings. An idea to achieve this goal may be the following. First we may try to develop a method to decompose arbitrary functions in terms

of *ranked functions* (under Definition 1). For instance, every distance $d(x, y)$ can be decomposed in the following two ranking functions:

$$f_1(x,y) = \begin{cases} d(x,y) & \text{if } x \leq y \\ 0 & \text{otherwise} \end{cases} \qquad \text{and} \qquad f_2(x,y) = \begin{cases} d(x,y) & \text{if } x \geq y \\ 0 & \text{otherwise} \end{cases}$$

by considering the supremum; i.e $d(x, y) = \sup\{f_1(x, y), f_2(x, y)\}$.

The second step would consist in generalizing the join strategy and the threshold computation according to the decomposition given in the previous step. For instance, considering the decomposition given above for arbitrary distances, the threshold for d would be given in terms of the threshold of f_1 and f_2.

References

1. Badr, M., Vodislav, D.: A general top-k algorithm for web data sources. In: Hameurlain, A., Liddle, S.W., Schewe, K.-D., Zhou, X. (eds.) DEXA 2011, Part I. LNCS, vol. 6860, pp. 379–393. Springer, Heidelberg (2011)
2. He, Z., Lo, E.: Answering why-not questions on top-k queries. IEEE Transactions on Knowledge and Data Engineering 99, 1 (2012)
3. Ilyas, I., Aref, W., Elmagarmid, A.: Supporting top-k join queries in relational databases. The VLDB Journal 13(3), 207–221 (2004)
4. Ilyas, I.F., Beskales, G., Soliman, M.A.: A survey of top-k query processing techniques in relational database systems. ACM Comput. Surv. 40(4), 11:1–11:58 (2008)
5. Luo, Y., Wang, W., Lin, X., Zhou, X., Wang, J., Li, K.: Spark2: Top-k keyword query in relational databases. IEEE Transactions on Knowledge and Data Engineering 23(12), 1763–1780 (2011)
6. Marian, A., Bruno, N., Gravano, L.: Evaluating top-k queries over web-accessible databases. ACM Trans. Database Syst. 29(2), 319–362 (2004)
7. Ranu, S., Singh, A.K.: Answering top-k queries over a mixture of attractive and repulsive dimensions. Proc. VLDB Endowment 5(3), 169–180 (2011)
8. Straccia, U.: Top-k retrieval for ontology mediated access to relational databases. Information Sciences 198, 1–23 (2012)
9. Straccia, U., Madrid, N.: A top-k query answering procedure for fuzzy logic programming. Fuzzy Set and Systems 205, 1–29 (2012)
10. Xin, D., Han, J., Chang, K.C.: Progressive and selective merge: computing top-k with ad-hoc ranking functions. In: Proceedings of the 2007 ACM SIGMOD International Conference on Management of Data, SIGMOD 2007, pp. 103–114. ACM, New York (2007)
11. Xu, C., Wang, Y., Lin, S., Gu, Y., Qiao, J.: Efficient fuzzy top-k query processing over uncertain objects. In: Bringas, P.G., Hameurlain, A., Quirchmayr, G. (eds.) DEXA 2010, Part I. LNCS, vol. 6261, pp. 167–182. Springer, Heidelberg (2010)
12. Yi, K., Li, F., Kollios, G., Srivastava, D.: Efficient processing of top-k queries in uncertain databases with x-relations. IEEE Transactions on Knowledge and Data Engineering 20(12), 1669–1682 (2008)